High-Level Synthesis with Geometric Algorithm Scheduling

Md. Tawhidur Rahman Tuhin
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
md-tawhidur-rahman.tuhin@stud.hshl.de

***Abstract*—High-Level Synthesis (HLS) is a technology that automatically converts C/C++ programs into hardware circuits. One of its most important tasks is deciding when each operation should be executed during processing. This scheduling problem is difficult because there are many possible ways to arrange the operations. This paper explains a method called GEM, which uses ideas from computational geometry to create efficient schedules. Instead of checking every possible solution, GEM matches operations with clock cycles in a smart and organized way. The method is faster than many traditional approaches while still producing high-quality schedules. The paper also introduces the basic mathematical concepts behind GEM, presents a C++ implementation of its core idea, and demonstrates its behavior using a standard benchmark example.**

***Index Terms*—High-Level Synthesis, Operation Scheduling, GEM Algorithm, Computational Geometry, Point Dominance, CDFG, RTL Design**

I. Introduction

As digital systems improve, creating hardware manually becomes more challenging. High-Level Synthesis (HLS) was designed to make this process easier by turning C/C++ specifications into RTL hardware designs [2]. The HLS process includes allocation, scheduling, and binding. Scheduling is one of the most critical steps because it determines the clock cycle assigned to each operation. The quality of scheduling directly influences hardware performance and resource usage.

There are some traditional scheduling techniques such as ASAP scheduling, ALAP scheduling, List Scheduling, Force-Directed Scheduling [4], and Integer Linear Programming (ILP). Some of them provide high-quality results, but they may require significant computational time. The resource-constrained version of the scheduling problem is NP-hard [5], so exact methods do not scale well.

GEM, introduced by Raje and Sarrafzadeh [1], does scheduling effectively by utilising ideas from computational geometry. It is a path-based algorithm: it takes whole dependency paths from the graph and places each path onto the schedule in one step, by solving a geometric matching problem called point dominance. Because a group of operations is scheduled together, the method is fast, and for each individual path the placement is optimal [1], [6]. This paper introduces GEM scheduling, describes its main concepts, and discusses its benefits, limitations, and practical implementation.

The paper is organized as follows. Section II explains the basics of HLS. Section III presents the geometric ideas behind GEM. Section IV describes the working principle step by step. Section V shows an example and a C++ implementation. Section VI discusses advantages and limitations, and Section VII concludes the paper.

II. High-Level Synthesis Basics

Before diving into geometric scheduling, it helps to understand the broader context: what HLS does and what input it receives.

A. From Algorithm to Hardware

An HLS tool takes a behavioral description—usually a C or C++ function—and translates it into an RTL design. The RTL design consists of functional units (adders, multipliers, comparators), registers that store intermediate results, multiplexers that route data, and a Finite-State Machine (FSM) that controls everything cycle by cycle [2], [3].

The transformation happens in three tightly coupled stages:

- **Allocation:** Decide how many of each hardware resource to use. For example, should the design include one multiplier or two? More resources allow more parallelism but cost more chip area.
- **Scheduling:** Assign each operation to a specific clock cycle, called a control step (or C-step). This determines how many cycles the design needs from input to output (the latency).
- **Binding:** Map each operation to a concrete hardware unit. For instance, if there are two multipliers and six multiply operations spread across four C-steps, binding decides which multiplier handles which multiplication.

These stages are interdependent—changing the allocation changes what schedules are feasible, and different schedules may lead to different binding choices.

B. The Control-Data Flow Graph

The central data structure used during scheduling is the Control-Data Flow Graph (CDFG), a directed graph $G = (V, E)$ where:

- Each vertex $v \in V$ represents one operation (e.g., a multiplication or addition).
- Each edge $(u, v) \in E$ means the output of operation u is an input to operation v, so u must finish before v can start.

- Each vertex v has a delay $d(v)$, the number of clock cycles the operation takes.

The longest path through the graph—the *critical path*—sets a lower bound on the schedule length, because operations along this path can never overlap, no matter how many resources are available.

C. ASAP, ALAP, and Mobility

Two simple traversals of the CDFG give useful bounds for each operation:

- **ASAP time** $\alpha(v)$: the earliest C-step at which v can possibly execute, respecting all data dependencies:
$$\alpha(v) = \max_{(u,v)\in E} \big(\alpha(u) + d(u)\big), \tag{1}$$
with $\alpha(v) = 1$ for operations without predecessors.
- **ALAP time** $\beta(v)$: the latest C-step at which v can start without exceeding a chosen maximum schedule length $L_{\max}$:
$$\beta(v) = \min_{(v,w)\in E} \big(\beta(w) - d(v)\big), \tag{2}$$
with $\beta(v) = L_{\max} - d(v) + 1$ for operations without successors.

The interval $[\alpha(v), \beta(v)]$ is called the *mobility window* of v. A wide window means the operation is flexible; a window of width zero means the operation is forced into a single C-step. Operations with zero mobility form the critical path, so a good scheduler pays special attention to them.

D. Why Scheduling Is Hard

Scheduling is easy when resources are unlimited: just use ASAP and run everything as soon as its inputs are ready. The problem becomes hard when there is a limited number of each resource type—for example, only two multipliers for six multiplications. Now the scheduler must spread operations across C-steps to avoid overloading any resource, while still keeping the total schedule as short as possible. This resource-constrained version is NP-hard for general graphs [5].

III. Geometric Scheduling Algorithm

The core idea behind GEM is elegant: rather than deciding one operation at a time, the algorithm takes a whole dependency path, turns the scheduling of that path into a geometry problem, and solves the geometry problem optimally [1].

A. Scheduling a Path as a Matching Problem

A *path* is a chain of operations where each operation directly depends on the previous one. The question "in which C-step should each operation of this path run?" can be viewed as a matching between two ordered lists [6]:

- List A: the operations of the path, in their dependency order.
- List B: the available C-steps, in order $1, 2, 3, \ldots$

Not every pairing is allowed. An operation can only be matched to a C-step where a suitable resource is still free, and it can only run after its predecessor in the path has finished. A valid schedule is a *non-crossing* matching: if operation u comes before operation v in the path, then u's C-step must be earlier than v's [6]. This is exactly the precedence constraint.

B. Mapping Possible Matchings to Points

GEM makes this matching problem geometric. Every *possible* assignment of an operation Op_i to a C-step c becomes one point in the x-y plane [1]:

$$x = \mathrm{index}(Op_i), \qquad y = c, \tag{3}$$

where $\mathrm{index}(Op_i)$ is the position of the operation inside the path (1 for the first operation, 2 for the second, and so on). If an operation has three feasible C-steps, it contributes three points, all with the same x-coordinate. The same construction is used by Memik et al. [6].

C. Point Dominance and Chains

The key geometric concept is *dominance*. A point p dominates a point q if and only if

$$X(p) > X(q) \quad \text{and} \quad Y(p) > Y(q), \tag{4}$$

where X and Y denote the coordinates of a point [1]. A *dominance chain* is a sequence of points where each point dominates the previous one—so the chain climbs strictly up and to the right.

Now the connection becomes clear. Picking one point per operation means choosing a C-step for every operation of the path. If the chosen points form a dominance chain, then later operations automatically get later C-steps—precedence is satisfied by the geometry itself. So scheduling the path optimally reduces to finding the best dominance chain of length k, where k is the number of operations in the path [1].

D. Weights and the Max-Weighted k-Chain

Each point also carries a weight that expresses how desirable that particular assignment is—for example, earlier C-steps get larger weights so the schedule stays short, and assignments that would require adding a new resource get very low weights [1], [6]. The problem then becomes: find the dominance chain of length k with the maximum total weight. This *max-weighted k-chain* problem can be solved in $\mathcal{O}(k \log k)$ time using the max-dominance algorithm of Atallah and Kosaraju [7]. Because the chain is found optimally, the placement of each path is provably the best possible placement for that path on the current partial schedule [1], [6].

IV. Working Principle: Step by Step

With the geometric ideas in place, the full GEM flow can be described in four steps [1].

A. Step 1 – Prepare the Graph

GEM first converts the CDFG into a directed acyclic graph (DAG). Loops are broken by removing their feedback edges; the loop condition is stored, and after scheduling, the loop body schedule is simply repeated as long as the condition holds. Conditional blocks need no special preprocessing either: scheduling proceeds as if there were no branches, and two operations from different branches of the same condition are even allowed to share the same C-step and resource, because only one branch executes at run time [1].

B. Step 2 – Prioritize the Paths

The algorithm extracts paths from the DAG and puts them in a queue. The longest path (the critical path) gets the highest priority. When two paths have the same length, the path that shares more operations with the previously queued path gets the higher priority—the authors call this the *commonness* principle [1]. The effect is that the most constrained operations, the ones with the least freedom, are scheduled first. Scheduling a wrong path first can cost extra C-steps, which is why this ordering matters.

C. Step 3 – Match Each Path Geometrically

The path at the head of the queue is scheduled onto the existing partial schedule. All feasible (operation, C-step) pairs are converted into weighted points as described in Section III, and the max-weighted k-chain gives the optimal placement of the whole path at once. The path is then removed from the queue and the next path is considered. Scheduling many operations in one step is also what makes GEM fast compared to algorithms that make a global search for every single operation [1].

D. Step 4 – Add Resources or C-Steps When Stuck

Sometimes no valid chain of length k exists—the path simply does not fit on the current partial schedule. What happens next depends on which problem GEM is solving [1]:

- **Timing-constrained mode:** the number of C-steps is fixed, so GEM adds a resource. The weights are set so that the cheapest resource type that resolves the conflict is added first.
- **Resource-constrained mode:** the resource budget is fixed, so GEM pushes the existing schedule down by one C-step. This "elongation" creates new candidate points for the path, and the matching is tried again.

The weights also give GEM a simple look-ahead: when two placements look equally good now, the one that leaves more freedom for future paths is preferred, so the algorithm does not only chase a local optimum [1].

Algorithm 1 summarizes the procedure.

E. Runtime and Quality

The overall time complexity of GEM is

$$T_{\mathrm{GEM}} = \mathcal{O}(n \cdot p \log p), \tag{5}$$

Algorithm 1 GEM scheduling flow, adapted from [1], Sec. 3, and [6], Fig. 7

Require: CDFG G, delays d, resource limits R
Ensure: Schedule σ

```
Convert G to a DAG (break loop edges, keep conditions)
Extract paths; queue by length, ties by commonness
while queue not empty do
    P ← head of queue
    Build weighted points for all feasible matchings of P
    σ_P ← max-weighted k-chain of the points
    if no chain of length k exists then
        Add cheapest helpful resource or push schedule down one C-step; retry
    end if
    Commit σ_P to the partial schedule; pop P
end while
return σ
```

where n is the number of disjoint paths in the CDFG and p is the number of nodes in the longest path [1]. The authors note that for a roughly square CDFG with m nodes, n is expected to be in the order of $\sqrt{m}$, so the method scales well. Loops and nested conditional branches cause no extra penalty in time complexity [1].

Regarding quality, each individual path is placed optimally, but the overall schedule is not guaranteed to be globally optimal: the greedy path order fixes decisions that a global method could still revise. Memik et al. [6] analyzed this local-vs-global gap for the same matching formulation and showed experimentally that the results stay close to the ILP optimum: in 9 of 14 tested benchmark graphs the matching-based scheduler found the exact optimal latency, while running in milliseconds where the ILP solver needed seconds to minutes.

V. Example and C++ Implementation

A. The HAL Differential-Equation Benchmark

To show GEM in action, consider the well-known differential-equation benchmark used by Paulin and Knight [4]. It computes one iteration of a numerical integration loop:

$$\begin{aligned} x_1 &= x + dx, \\ u_1 &= u - (3 \cdot x \cdot u \cdot dx) - (3 \cdot y \cdot dx), \\ y_1 &= y + u \cdot dx, \\ c &= x_1 < a. \end{aligned}$$

The product $3 \cdot x \cdot u \cdot dx$ is computed in a balanced way as $(3 \cdot x) \cdot (u \cdot dx)$, so the CDFG contains six multiplications (M_1–M_6), two subtractions (S_1, S_2), one addition for y_1 (A_1), one addition for x_1 (A_2), and one comparison (C_1). Fig. 1 shows the resulting graph. We assume unit delays, a budget of two multipliers, and one adder, one subtractor, and one comparator—the same setup for which the original paper reports its result [1].

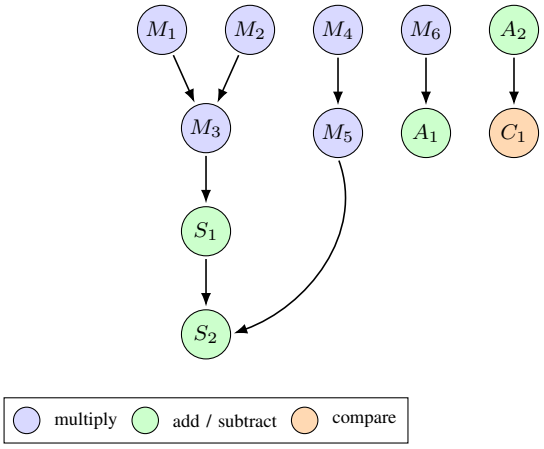

Fig. 1. CDFG of the HAL differential-equation benchmark with the balanced multiplication tree. Redrawn by the author after the example introduced in [4], which is also used as the "DIFF. EQ." benchmark in [1], Table A. The critical path $M_1 \to M_3 \to S_1 \to S_2$ has length four.

TABLE I
MOBILITY WINDOWS FOR THE HAL DIFFERENTIAL-EQUATION BENCHMARK FROM [4] ($L_{\max} = 4$; VALUES COMPUTED BY THE AUTHOR, SCHEDULE RESULT REPORTED IN [1], TABLE A).

Op	Meaning	ASAP	ALAP	Mobility
M_1	$3 \cdot x$	1	1	0
M_2	$u \cdot dx$	1	1	0
M_3	$M_1 \cdot M_2$	2	2	0
M_4	$3 \cdot y$	1	2	1
M_5	$M_4 \cdot dx$	2	3	1
M_6	$u \cdot dx$	1	3	2
S_1	$u - M_3$	3	3	0
S_2	$S_1 - M_5$	4	4	0
A_1	$y + M_6$	2	4	2
A_2	$x + dx$	1	3	2
C_1	$A_2 < a$	2	4	2

With $L_{\max} = 4$, the ASAP and ALAP traversals give the mobility windows in Table I. The chain $M_1 \to M_3 \to S_1 \to S_2$ has zero mobility everywhere: it is the critical path.

GEM schedules the critical path $M_1 \to M_3 \to S_1 \to S_2$ first, placing it in C-steps 1–4. The remaining paths follow in priority order and are matched onto the partial schedule: M_2 joins C-step 1 (second multiplier), M_4 and M_5 land in C-steps 2 and 3, M_6 moves to C-step 3 because both multipliers are busy earlier, which places A_1 in C-step 4, and finally A_2 and C_1 take C-steps 1 and 2. Fig. 2 shows the geometric matching for the path $M_6 \to A_1$ as an example: infeasible pairings drop out, and the max-weighted dominance chain through the remaining points gives the assignment. No C-step exceeds any resource limit, and the final schedule needs four C-steps with two multipliers and one unit per ALU operation class. This matches the result reported for GEM on this benchmark, which equals the best result of the compared classical methods [1].

B. C++ Implementation of the Core Idea

Listing 1 shows a compact C++17 implementation of the per-path scheduling step. To keep the code short and readable,

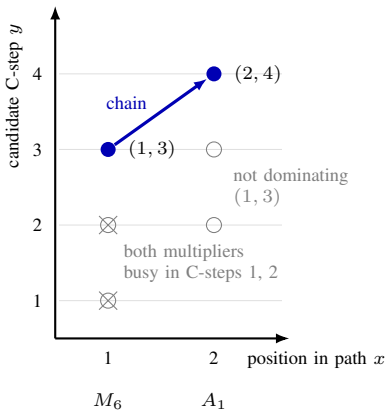

Fig. 2. GEM's geometric matching for the path $M_6 \to A_1$ on the partial schedule. The construction follows [1], Figs. 10–11, and [6], Fig. 4(b): every feasible (operation, C-step) pair becomes a point with x = position in the path and y = the C-step; the example values are computed by the author for the HAL schedule. Crossed points offer no free multiplier; gray points do not dominate the choice for M_6. The max-weighted dominance chain (arrow) assigns M_6 to C-step 3 and A_1 to C-step 4.

it uses a simplified greedy rule instead of the full weighted k-chain: every operation of the path is matched to the earliest C-step inside its window that still has a free resource. For a latency-only weight function this earliest-fit rule selects the same chain that the max-weighted k-chain would select, because earlier C-steps carry larger weights [1], [6]. The full algorithm would replace the greedy loop with the $\mathcal{O}(k \log k)$ max-dominance procedure of Atallah and Kosaraju [7].

```cpp
#include <vector>
#include <optional>

struct Op {
  int id;      // unique identifier
  int asap;    // earliest allowed C-step
  int alap;    // latest allowed C-step
  int delay;   // duration in clock cycles
  int type;    // resource type index
};

// free[t][c] = number of free units of
// resource type t in C-step c.
using Free = std::vector<std::vector<int>>;

// Schedules one dependency path onto the
// partial schedule. Returns the C-step of
// every operation, or nullopt if the path
// does not fit (GEM then adds a resource
// or pushes the schedule down one C-step).
std::optional<std::vector<int>>
schedulePath(const std::vector<Op>& path,
             Free& free) {
  std::vector<int> sigma(path.size());
  int cstep = 1;
  for (size_t i = 0; i < path.size(); ++i) {
    const Op& op = path[i];
    if (cstep < op.asap) cstep = op.asap;
    // find earliest C-step with a free unit
    while (cstep <= op.alap &&
           free[op.type][cstep] == 0)
      ++cstep;
    if (cstep > op.alap)
      return std::nullopt;   // no k-chain
    --free[op.type][cstep];  // occupy unit
    sigma[i] = cstep;
    cstep += op.delay;       // successor
  }                          // starts later
  return sigma;
```

TABLE II
COMPARISON OF HLS SCHEDULING APPROACHES.

Method	Runtime	Quality guarantee	Source
ASAP / ALAP	$\mathcal{O}(\|V\| + \|E\|)$	bounds only	[2]
List Scheduling	low (heuristic)	none	[5]
Force-Directed	$\mathcal{O}(\|V\|^2)$	none	[4]
ILP	exponential	globally optimal	[5]
GEM	$\mathcal{O}(np \log p)$	optimal per path	[1]

```
40 }
```

Listing 1. Per-path scheduler (simplified earliest-fit version of GEM's matching step).

For the HAL benchmark, calling this routine path by path in the priority order of Section IV reproduces the four-cycle schedule described above. The loop body is linear in the path length, so the cost per path stays small; in the full algorithm the matching step dominates with $\mathcal{O}(p \log p)$ per path, giving the total of $\mathcal{O}(np \log p)$ [1].

VI. Advantages and Limitations

A. Advantages

- **Fast.** The total runtime is $\mathcal{O}(np \log p)$, which is far below the exponential cost of ILP. The matching-based approach needed only milliseconds on benchmark graphs where an ILP solver ran for seconds to minutes [1], [6].
- **Schedules whole paths at once.** Classical heuristics such as list scheduling or force-directed scheduling decide one operation at a time [4]. GEM places a complete dependency chain in a single optimal step, which is both a speed and a quality advantage [1].
- **Per-path optimality.** The max-weighted k-chain is provably the best placement of the path on the current partial schedule [1], [7]. Most fast heuristics offer no such guarantee at all.
- **Loops and branches for free.** Loops are broken into a DAG and repeated after scheduling; conditional branches may even share resources across mutually exclusive paths. Neither feature increases the time complexity [1].
- **Flexible objectives.** Because the goals are encoded in point weights, the same machinery can optimize latency, resource cost, the use of special hardware blocks, or the distribution of slack, simply by changing the weight function [6].

B. Limitations

- **The guarantee stops at the path.** Each path is placed optimally, but the combination of paths is decided greedily. The final schedule can therefore end up slightly above the true optimum on irregular graphs; Memik et al. observed this gap in 5 of 14 benchmarks [6].
- **Path order matters.** A wrongly chosen path order can cost extra C-steps. The longest-path and commonness rules reduce this risk but do not remove it [1].
- **No loop pipelining.** GEM schedules one loop iteration and repeats it. Modern HLS tools overlap loop iterations to increase throughput, and the geometric formulation does not directly cover this.
- **Limited industrial uptake.** Commercial HLS flows rely on tuned list-scheduling and ILP variants combined with many other optimizations, so the per-path guarantee is hard to turn into a visible advantage on full industrial designs.

VII. Conclusion

High-Level Synthesis makes hardware design faster and more accessible by automating the translation from C/C++ to RTL. Scheduling is the toughest part of this automation because the resource-constrained problem is NP-hard [5].

GEM takes a fresh angle on this challenge. It treats every dependency path as one unit, turns all feasible operation-to-C-step assignments into weighted points on a plane, and picks the best schedule for the path as a maximum-weighted dominance chain in $\mathcal{O}(k \log k)$ time [1], [7]. The complete algorithm runs in $\mathcal{O}(np \log p)$, handles loops and conditional branches without extra cost, and delivers schedules that match or come close to the exact optimum on standard benchmarks [1], [6]. On the classical differential-equation benchmark it reaches the four-cycle schedule with two multipliers, which equals the best known result for that setup [1].

The main open challenges are extending the optimality guarantee from single paths to complete graphs and combining the geometric formulation with modern HLS features such as loop pipelining. The flexible weight function already points in a promising direction, since later work has used the same matching core to plan slack and to exploit special hardware blocks during scheduling [6].

References

[1] S. Raje and M. Sarrafzadeh, "GEM: A geometric algorithm for scheduling," in *Proc. IEEE Int. Symp. Circuits and Systems (ISCAS)*, 1993, pp. 1991–1994.

[2] D. D. Gajski, N. D. Dutt, A. C.-H. Wu, and S. Y.-L. Lin, *High-Level Synthesis: Introduction to Chip and System Design*. Boston, MA: Kluwer Academic, 1992.

[3] P. Coussy and A. Morawiec, Eds., *High-Level Synthesis: From Algorithm to Digital Circuit*. Dordrecht: Springer, 2008.

[4] P. G. Paulin and J. P. Knight, "Force-directed scheduling for the behavioral synthesis of ASIC's," *IEEE Trans. Computer-Aided Design*, vol. 8, no. 6, pp. 661–679, Jun. 1989.

[5] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York: McGraw-Hill, 1994.

[6] S. O. Memik, R. Kastner, E. Bozorgzadeh, and M. Sarrafzadeh, "A scheduling algorithm for optimization and early planning in high-level synthesis," *ACM Trans. Design Automation of Electronic Systems*, vol. 10, no. 1, pp. 33–57, Jan. 2005.

[7] M. J. Atallah and S. R. Kosaraju, "An efficient algorithm for maxdominance, with applications," *Algorithmica*, vol. 4, no. 2, pp. 221–236, 1989.